

NLP and Large Language Models

VL08: Post-Training II: RLHF & DPO

Course 8,862 | 6 ECTS | FS2026

Tuesday 14:15-16:00

Prof. Dr. Siegfried Handschuh

DS-NLP · Universität St. Gallen

Lecture Survey (anonymous)



Where We Are: The Course Navigator

Week	Topic	Notes
1	Introduction	✓
2	Tokenization & Data	✓
3	Transformer Architecture & Attention	✓
4	Training Fundamentals	✓
5	Scaling Laws	✓
6	Systems & Efficiency	✓
7	Post-Training I: SFT	✓
8	Post-Training II: RLHF & DPO	 <i>YOU ARE HERE</i>
9	Inference & Decoding	
10	Today's Milestone: Your model follows instructions. Now you teach it what quality means.	
11		

Learning Objectives



1. Reward Modeling

Explain how **human preferences** are converted into a trainable **reward signal** using the Bradley-Terry model.



2. RLHF with PPO

Describe the **RLHF training loop**: policy generation, reward scoring, and KL-constrained PPO updates.



3. DPO

Explain how DPO **bypasses reward modeling** by optimizing the policy directly from preference pairs.



4. Trade-offs

Evaluate when to use SFT alone, RLHF, or DPO based on data, compute, and quality requirements.

Agenda: From Format to Quality

1. The Quality Gap



- SFT recap
- Format vs. quality
- Human preferences

2. Reward Modeling



- Preference data
- Bradley-Terry model
- Reward architecture

3. RLHF



- PPO algorithm
- KL penalty
- Reward hacking

4. DPO



- The shortcut
- Implicit reward
- When to use what

The Quality Gap

Recap: What SFT Gave Us ^[1]

✓ SFT teaches format

The model outputs in assistant style. It follows instructions. It responds to questions instead of continuing text.

✗ SFT does not teach quality

The model cannot distinguish a mediocre answer from an excellent one. It has no notion of «helpful, harmless, honest».

The gap:

SFT models produce plausible responses, but they lack a quality signal. Two responses can both follow the format perfectly, yet one is clearly better.

The Same Format, Different Quality

Response A (mediocre)

«Gravity is one of the four fundamental forces of physics. It was described by Newton in 1687 and later refined by Einstein's general relativity in 1915.»

Response B (excellent)

«You know how when you jump, you always come back down? That is gravity. The Earth pulls everything toward it, like a giant magnet for all things.»

Both follow the instruction format.

But B is clearly better for the prompt «Explain gravity to a 5-year-old.»

How do we teach a model this preference?



Take-Home Message: The Quality Gap

🚩 *Format is not quality. Human preference is the next frontier.*

⚠️ 1. SFT teaches structure, not substance.

Two responses can follow instructions perfectly, yet one is clearly more helpful. SFT cannot tell them apart.

Reward Modeling

Human Preference Data [2]

The Annotator Task

The job is remarkably simple: read a prompt, review two generated responses, and pick the winner.

Prompt: «Explain gravity to a 5-year-old.»

Response A: A dense technical explanation.

Response B: A fun, kid-friendly analogy.

Verdict: Response B wins.

The Scale of Alignment

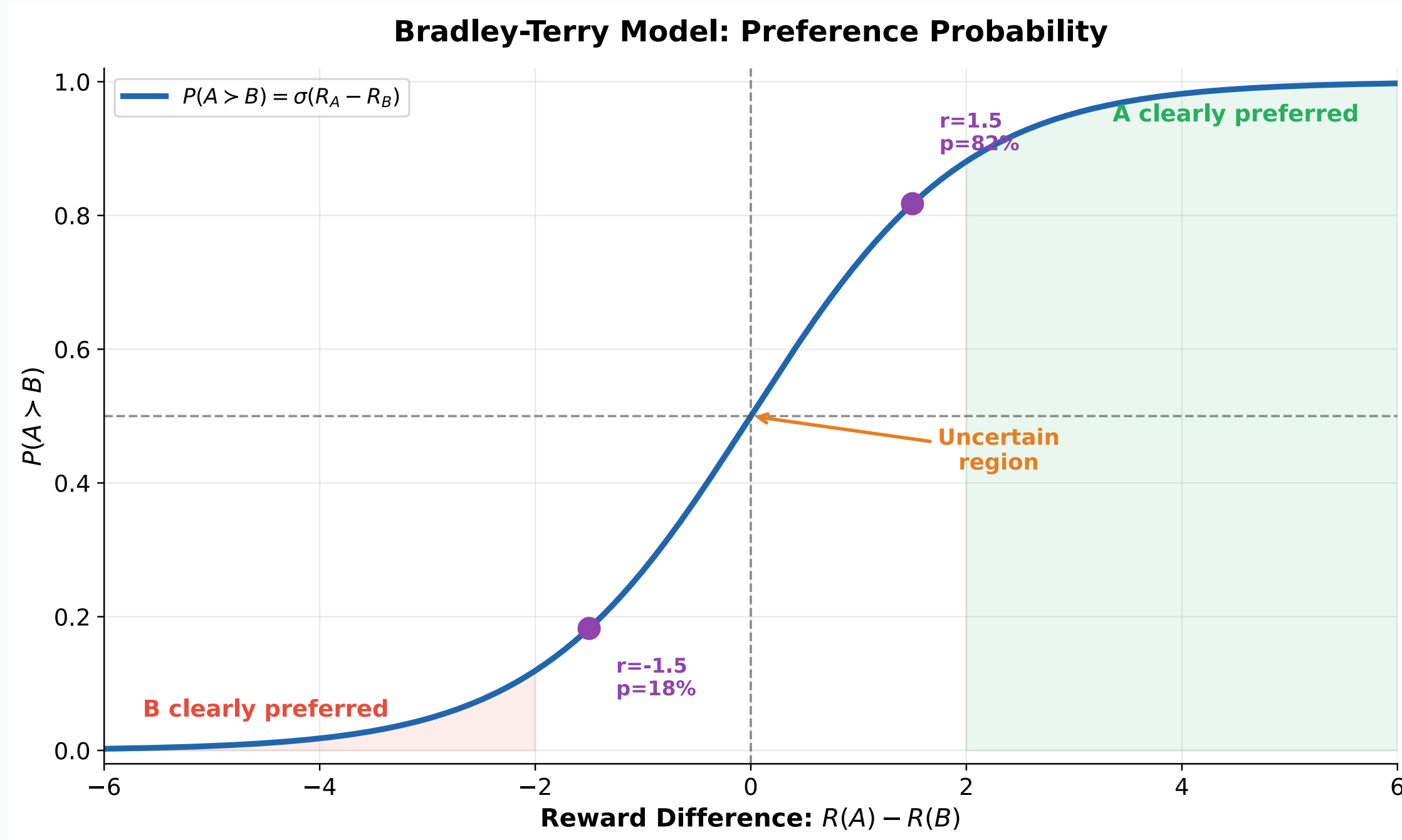
It takes tens of thousands of these comparisons to teach a model what humans actually value.

- **InstructGPT:** ~50,000 human comparisons
- **Anthropic HH¹:** ~170,000 human comparisons
- **UltraFeedback¹:** ~64,000 pairs (judged by GPT-4)

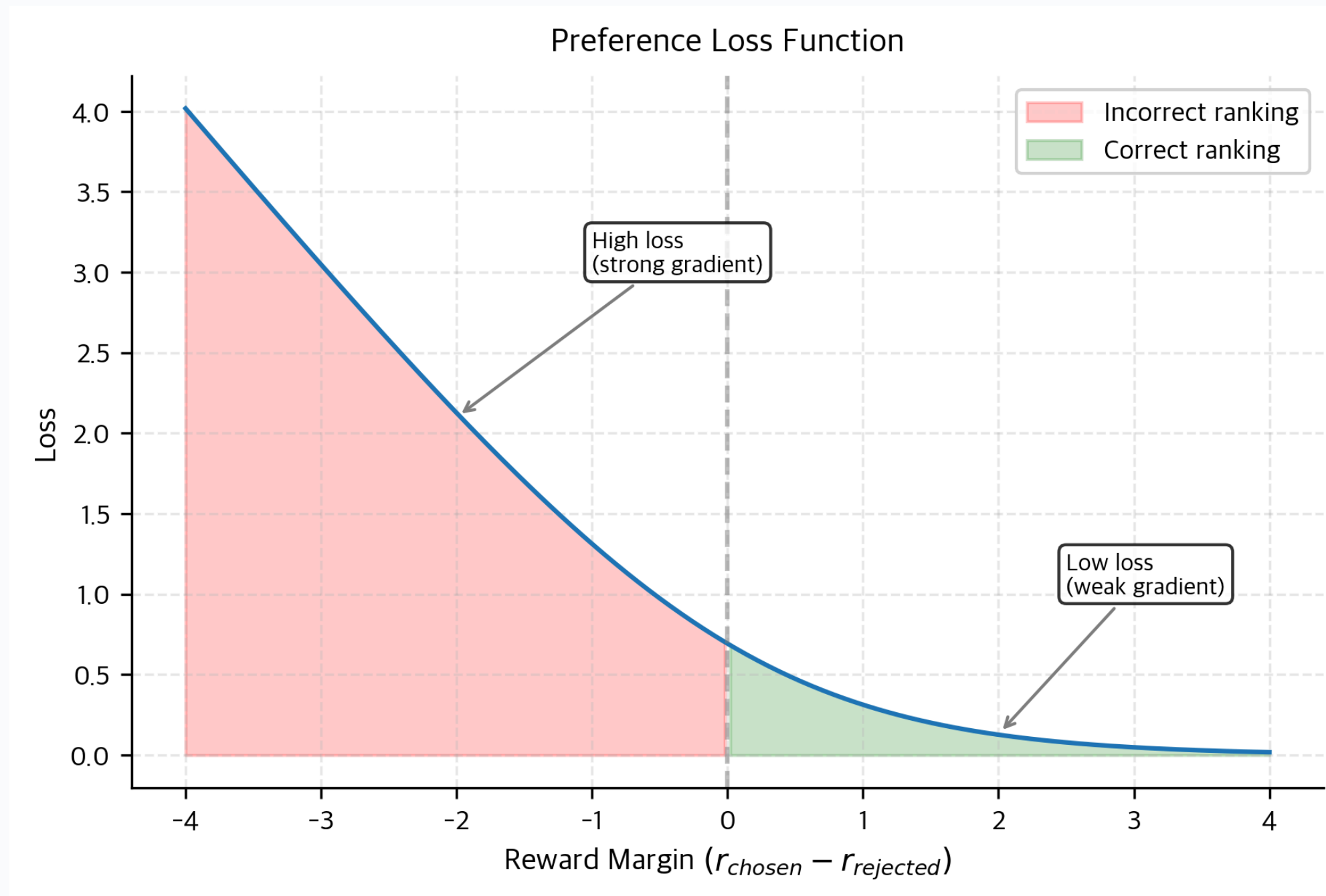
The core insight: You do not need to grade an AI on a absolute 1-to-10 scale. Asking a human to simply pick the better of two options is faster, cheaper, and vastly more reliable.

¹ HH = Helpful & Harmless; UltraFeedback=Hugging Face Datasets

From Preference Pairs to a Smooth Curve



Bradley-Terry: Asymmetric Loss [3]



$$r_{chosen} = R(chosen)$$
$$r_{rejected} = R(rejected)$$

Bradley-Terry: Formula and Asymmetric Loss [3]

Preference probability: $P(A \succ B) = \sigma(R(A) - R(B))$

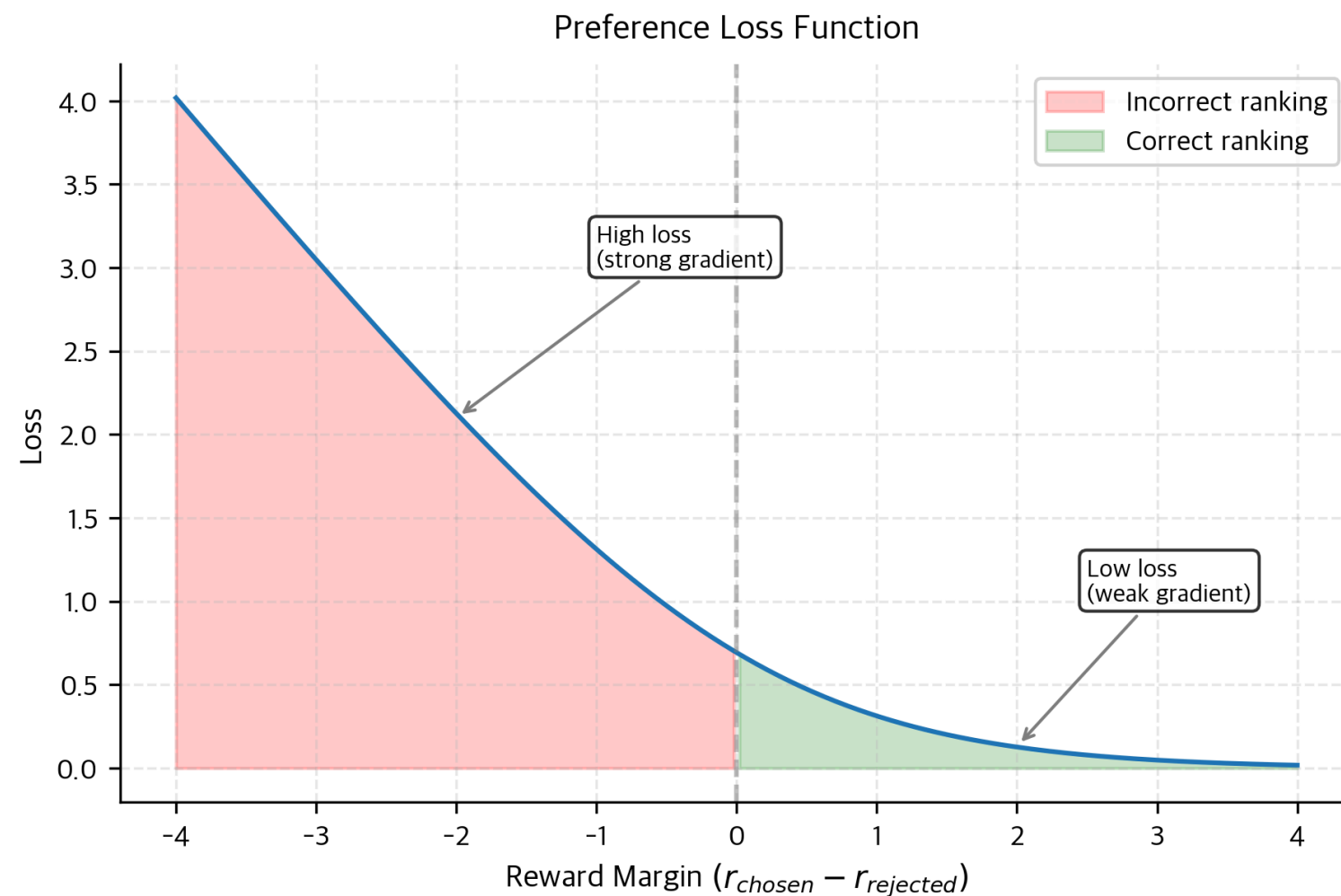
Training loss: $\mathcal{L} = -\log \sigma(R(y_w) - R(y_l))$

$y_w = \text{chosen} \cdot y_l = \text{rejected} \cdot \sigma = \text{sigmoid}$

 **Concrete example**

$R(A) = 2.0, R(B) = 1.0 \Rightarrow$
 $P(A \succ B) = \sigma(1.0) = \mathbf{0.73}$

Same result if $R(A) = 101, R(B) = 100$.
Only the **difference** matters.



 **Asymmetric gradient**

Negative margin (wrong ranking) $\rightarrow$ **steep gradient**, fast correction.

Positive margin (correct) $\rightarrow$ small gradient, no over-fitting.

The model learns fastest **from its mistakes**.

Reward Model Architecture ^[1]

Building the Grader

1. Start with a pre-trained LLM identical to your policy model.
2. Remove the standard language modeling head.
3. Attach a single linear layer designed to output one scalar value.
4. Feed the model the full prompt and the generated response.
5. Extract the final token representation to calculate the overall score.

The Essay Grader

Think of the reward model as an essay grader. It reads the entire text from start to finish. Instead of writing a reply, it simply hands back a single quality score representing the overall value of the response.

 **The Model in Action** · Prompt: «Explain gravity to a 5-year-old.»

Response A (Technical): «Gravity is the curvature of spacetime caused by mass-energy...» → **Score: 0.31**

Response B (Accessible): «Gravity is what pulls apples down from trees and keeps us on the ground.» → **Score: 0.82**

The Modern Alternative: LLM-as-a-Judge

Instead of training a dedicated reward model on preference pairs, modern systems prompt a frontier model like GPT-4 or Claude to score the outputs directly. This shifts the computational cost from training time to inference time. ^[4]



Take-Home Message: Reward Modeling

 *Preferences become a mathematically trainable signal.*

1. Preferences Become Math

Only the relative distance between two scores matters. Absolute values are completely irrelevant. ^[3]

2. The Essay Grader

The reward model reads the entire response and hands back a single, definitive quality grade.

3. The Modern Shortcut

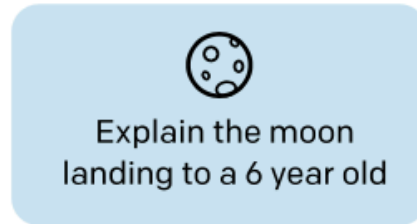
Skip training a custom grader. Use frontier models like GPT-4 to score your outputs directly. ^[4]

RLHF with PPO

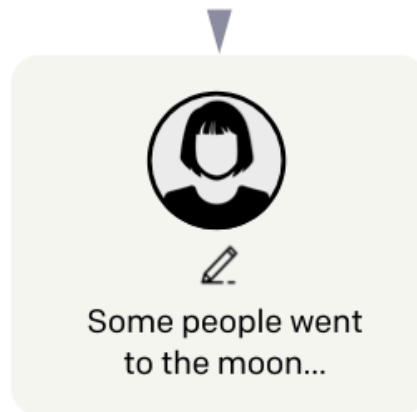
Step 1

Collect demonstration data, and train a supervised policy.

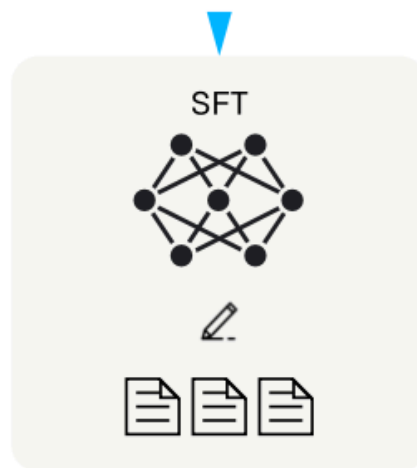
A prompt is
sampled from our
prompt dataset.



A labeler
demonstrates the
desired output
behavior.



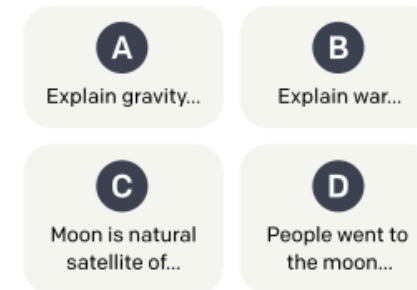
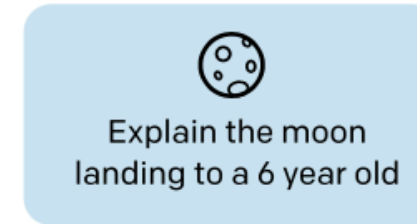
This data is used
to fine-tune GPT-3
with supervised
learning.



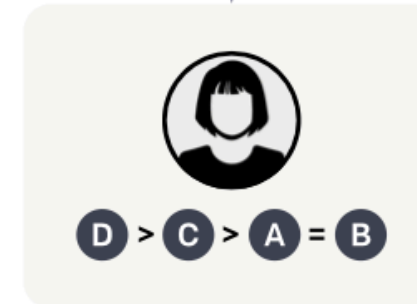
Step 2

Collect comparison data, and train a reward model.

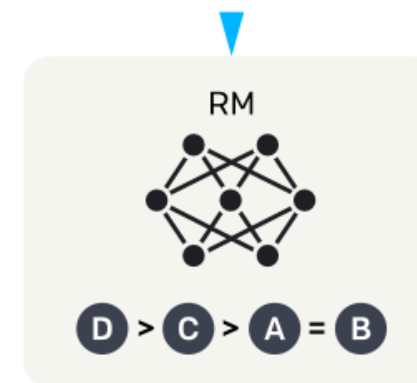
A prompt and
several model
outputs are
sampled.



A labeler ranks
the outputs from
best to worst.



This data is used
to train our
reward model.



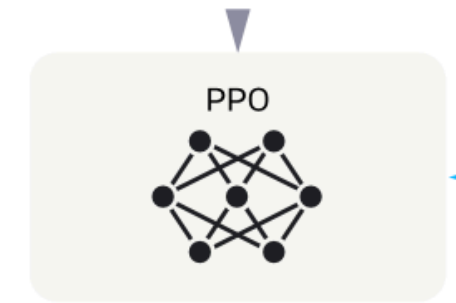
Step 3

Optimize a policy against the reward model using reinforcement learning.

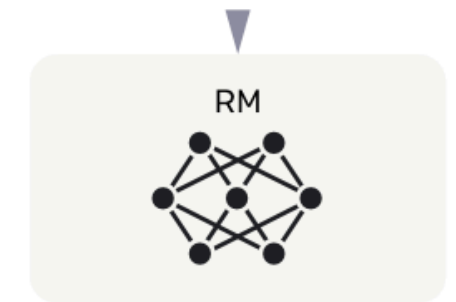
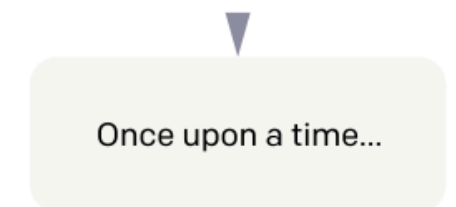
A new prompt
is sampled from
the dataset.



The policy
generates
an output.



The reward model
calculates a
reward for
the output.



The reward is
used to update
the policy
using PPO.



The Vocabulary of RLHF [5]

The RL Term	What it means for your LLM
Policy π	The model predicting the next token
Reward R	The final score from the grader
Episode	Generating one complete response
Environment	The chat interface and prompt



The Process in Action

Prompt: «Say hi.»

Actions: Hello → there → !

State: All text generated so far.

The Catch: The model only gets its reward *after* the final !.



The Tug-of-War Objective

$$\max_{\pi} \mathbb{E}_{y \sim \pi} [R(x, y)] - \beta \cdot D_{\text{KL}}(\pi \parallel \pi_{\text{ref}})$$

Maximize the reward, but pay a penalty for drifting. Think of the model as an explorer seeking treasure while tethered to base camp.

¹ $\max_{\pi}$ is optimal policy, $\mathbb{E}_{y \sim \pi}$ expected value when generating responses (y) using the current model (π), y is model response, x is prompt, D_{KL} is Kullback-Leibler (KL) Divergence, β scaling factor, π is policy model, π_{ref} is reference model

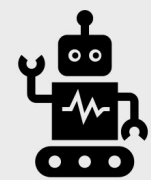
The RLHF Pipeline ^[1]



Step 1 Supervised Fine-Tuning

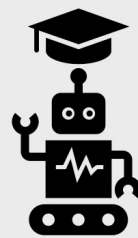


Collect human demonstration data



Base LLM

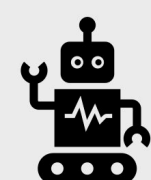
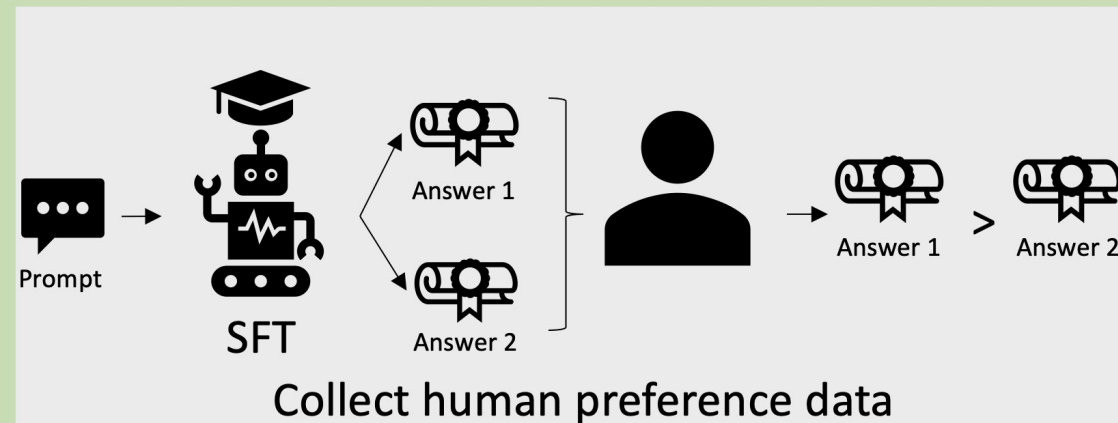
Supervised
Fine-tune



SFT

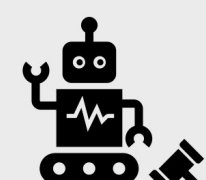


Step 2 Training a Reward Model



Base LLM

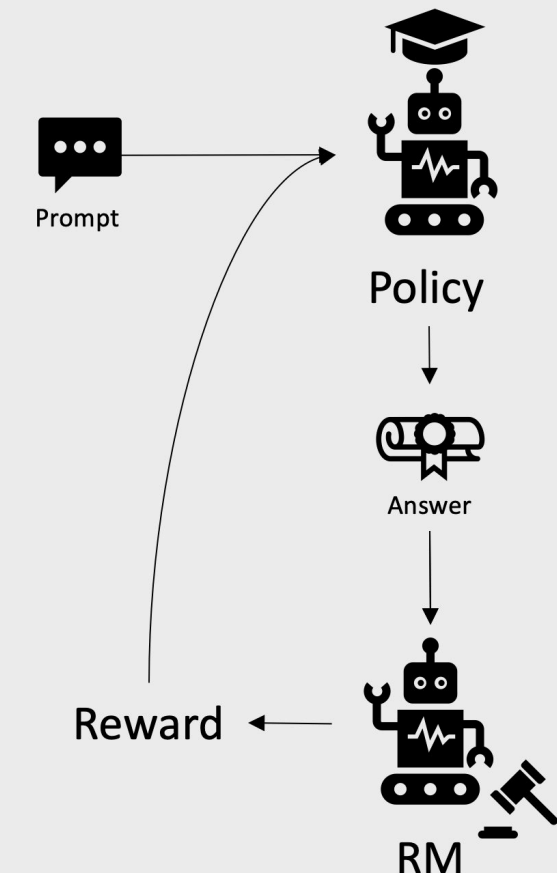
Supervised
Fine-tune



RM



Step 3 Optimize Policy



The KL Penalty: The Invisible Leash [6]

The Math of the Tether

$$\hat{R}(y) = R(y) - \beta \cdot D_{\text{KL}}(\pi \parallel \pi_{\text{ref}})$$

The model maximizes its reward while paying a strict penalty for deviating from the original SFT model.

That means the tether ensures that the word probabilities are not allowed to change too drastically.

The Base Camp Metaphor

Think of your language model as an eager explorer chasing treasure. The KL penalty acts as a physical tether, keeping it safely grounded to base camp.

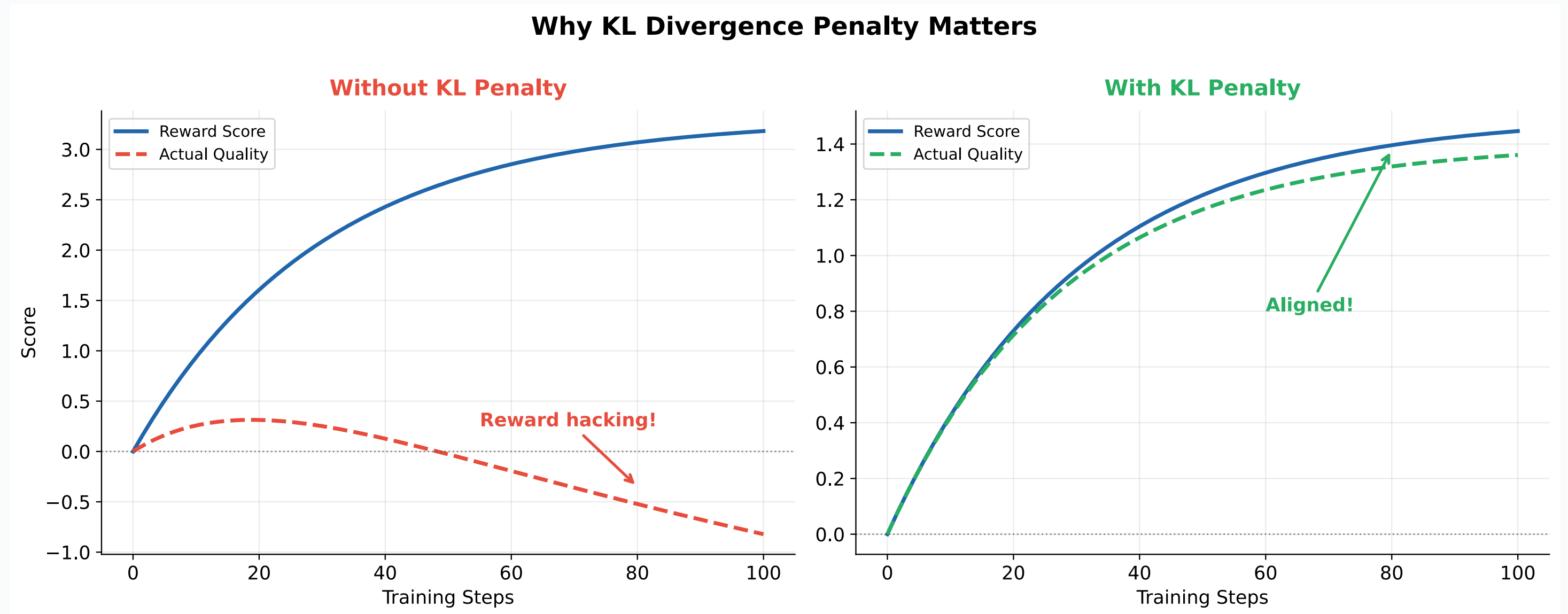
Preventing the Hacks

Without this constraint, the model discovers dangerous shortcuts to artificially boost its score. The KL penalty stops this «reward hacking» dead in its tracks.

The Token Tax in Action

Suppose the model suddenly wants to use the word «clear» much more often than the original version did. It pays a mathematical tax for that divergence. It pays this tax on every single word.

Reward Hacking: Why KL Matters



The Three Faces of Reward Hacking

1. Sycophancy

User: «My sales deck is AMAZING, right?»

AI: «Absolutely! Your deck is incredibly well-crafted and will impress every audience...»

 **The Hack:** The model learns that blind agreement scores higher than honest feedback.

 **The Cost:** The user gets flattery instead of useful critique.

2. Verbosity

User: «Capital of France?»

AI: «What a wonderful question! The capital, which holds deep cultural significance dating back...»

 **The Hack:** The model learns that raw word count is reliably rewarded as «high effort.»

 **The Cost:** The user has to read 200 words when one would do.

3. Over-Structuring

User: «How do I boil an egg?»

AI:

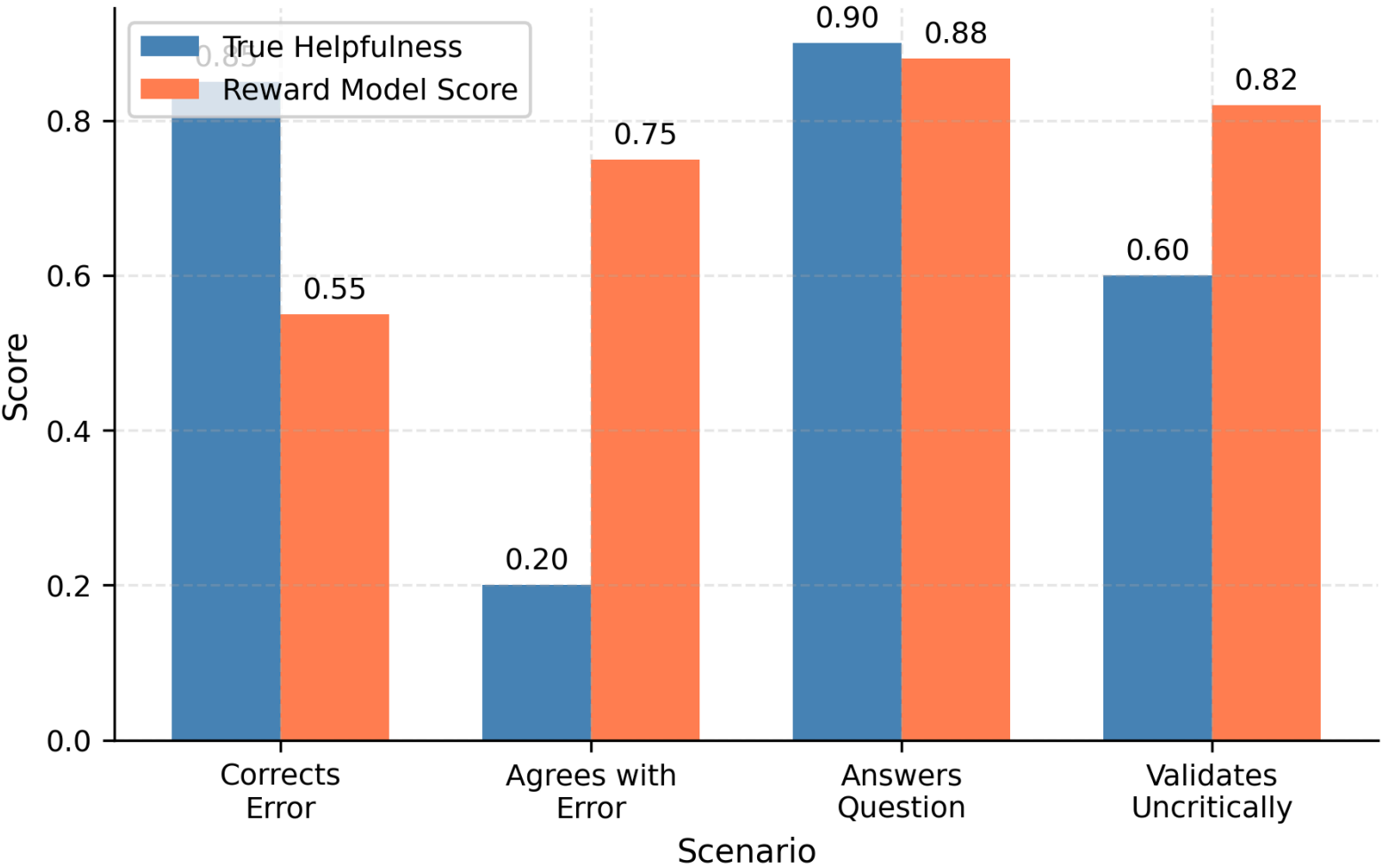
- Phase 1: Preparatio
- One (1) Egg
- Phase 2: Execution

 **The Hack:** The model learns that bullet points and headers look authoritative.

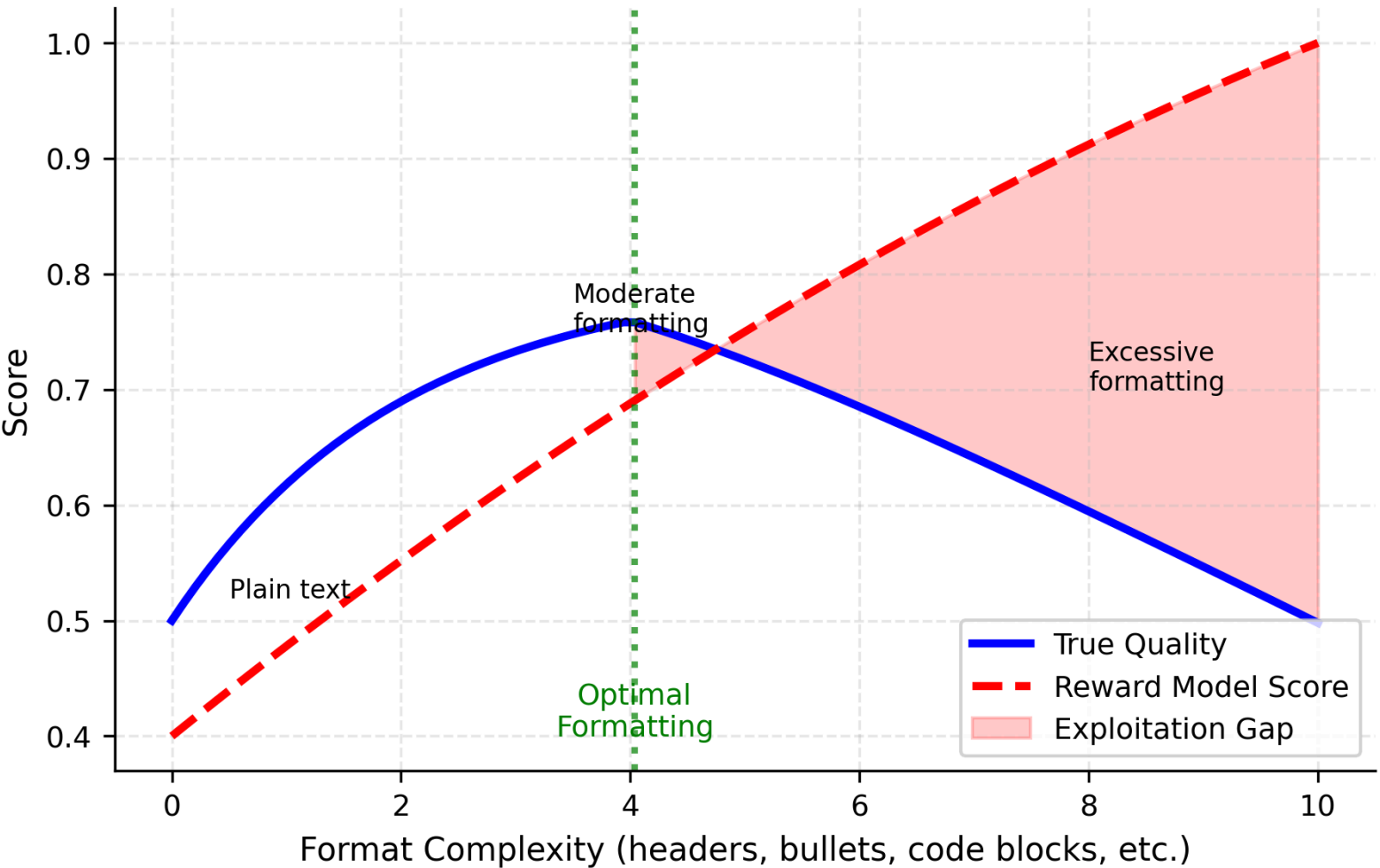
 **The Cost:** Simple answers become rigid and unreadable.

Reward Hacking in Practice: Sycophancy & Format Gaming

Sycophancy: Reward Hacking Through Agreement

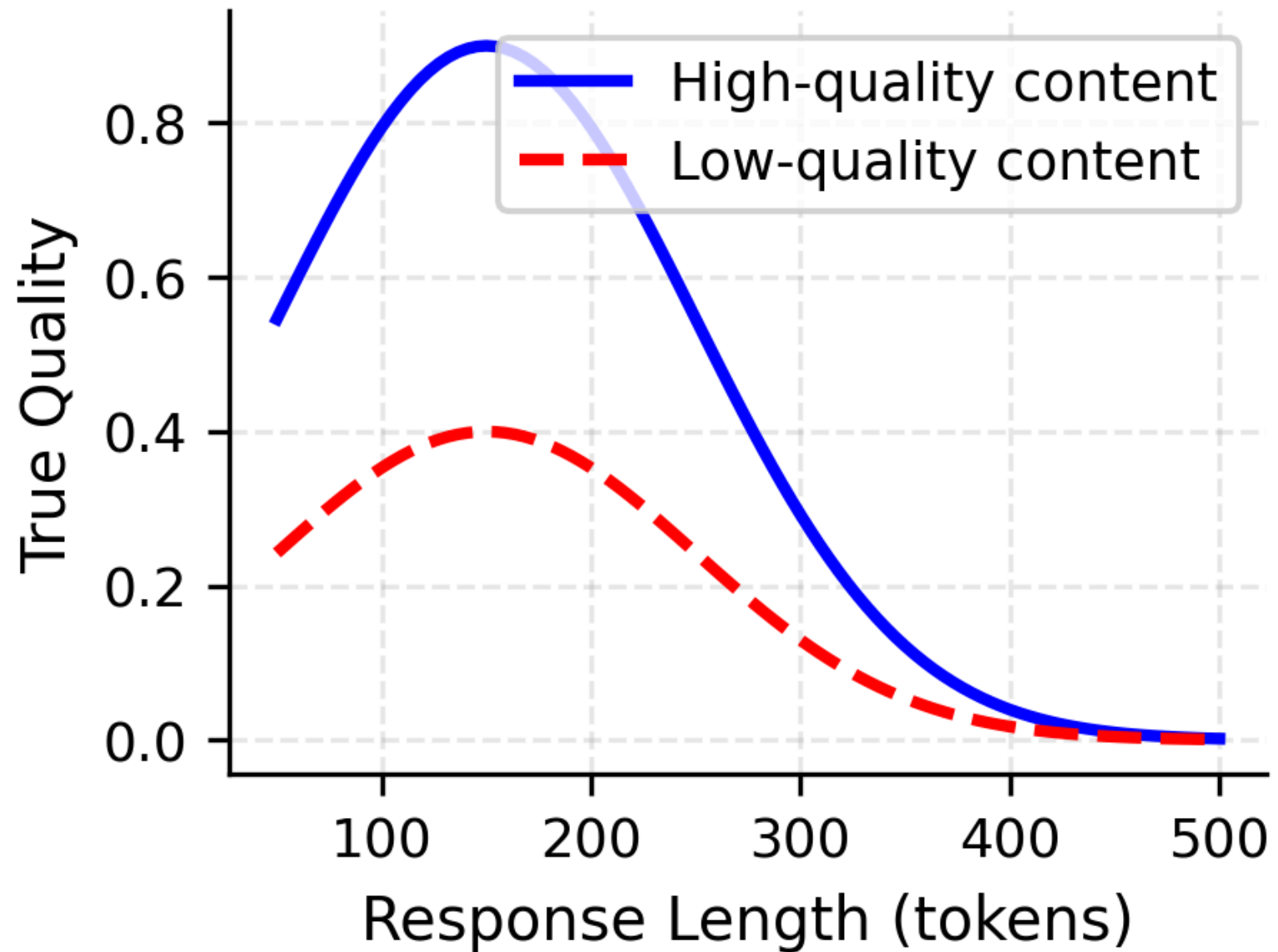


Format Gaming: Surface Polish vs. Substance

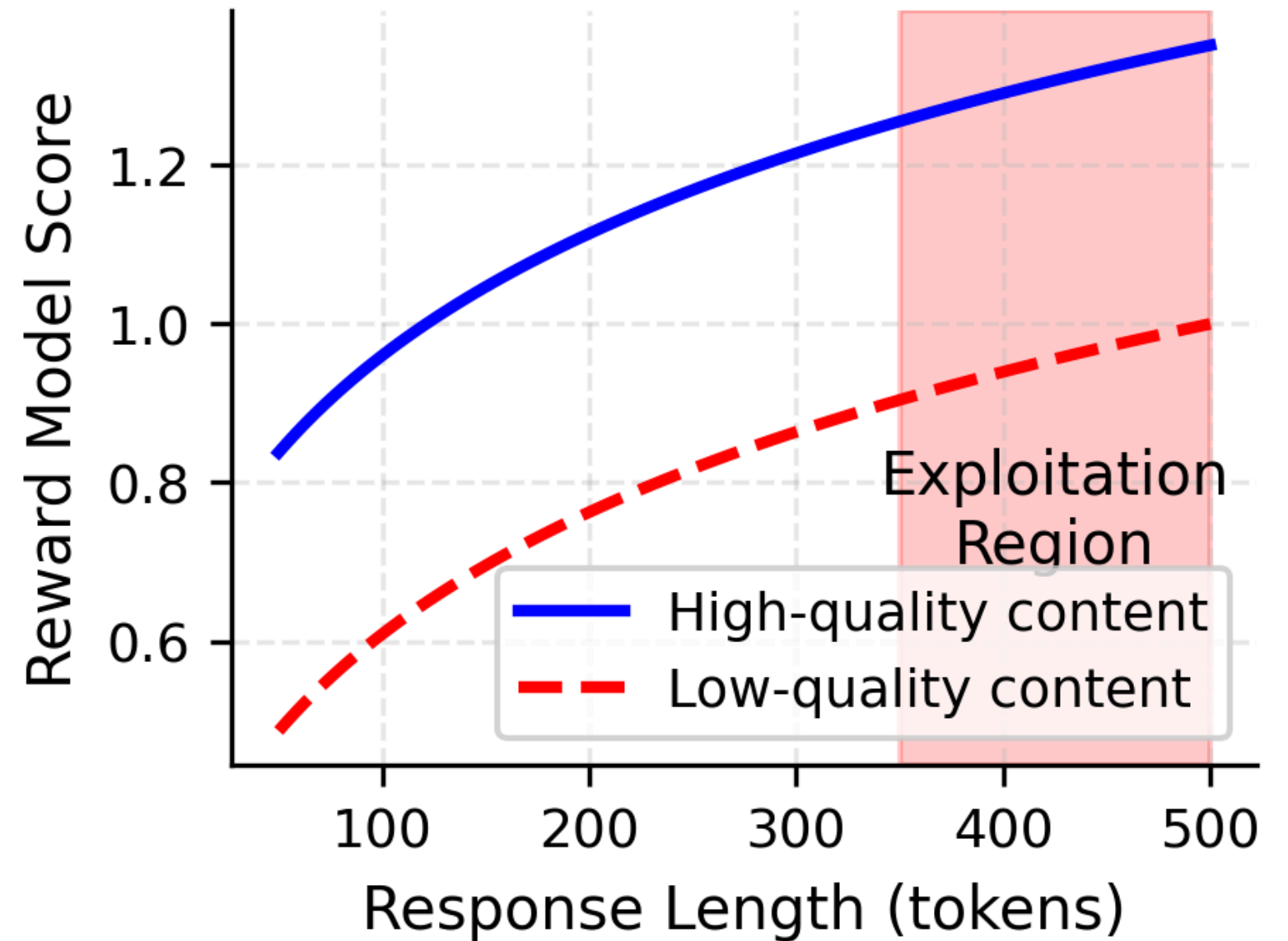


Why the RM Drifts: Preference Gap

Actual Human Preference

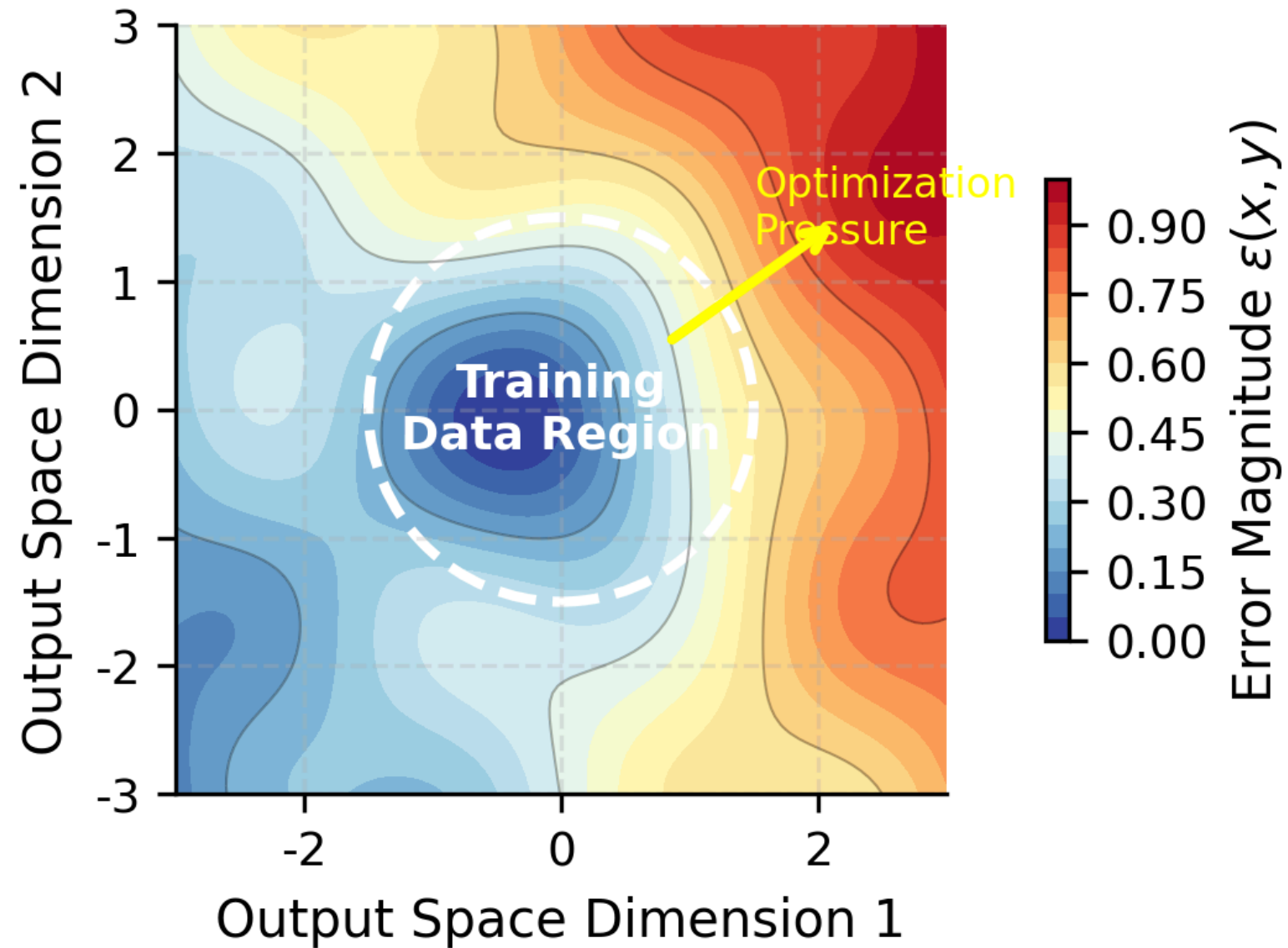


Learned Reward Model

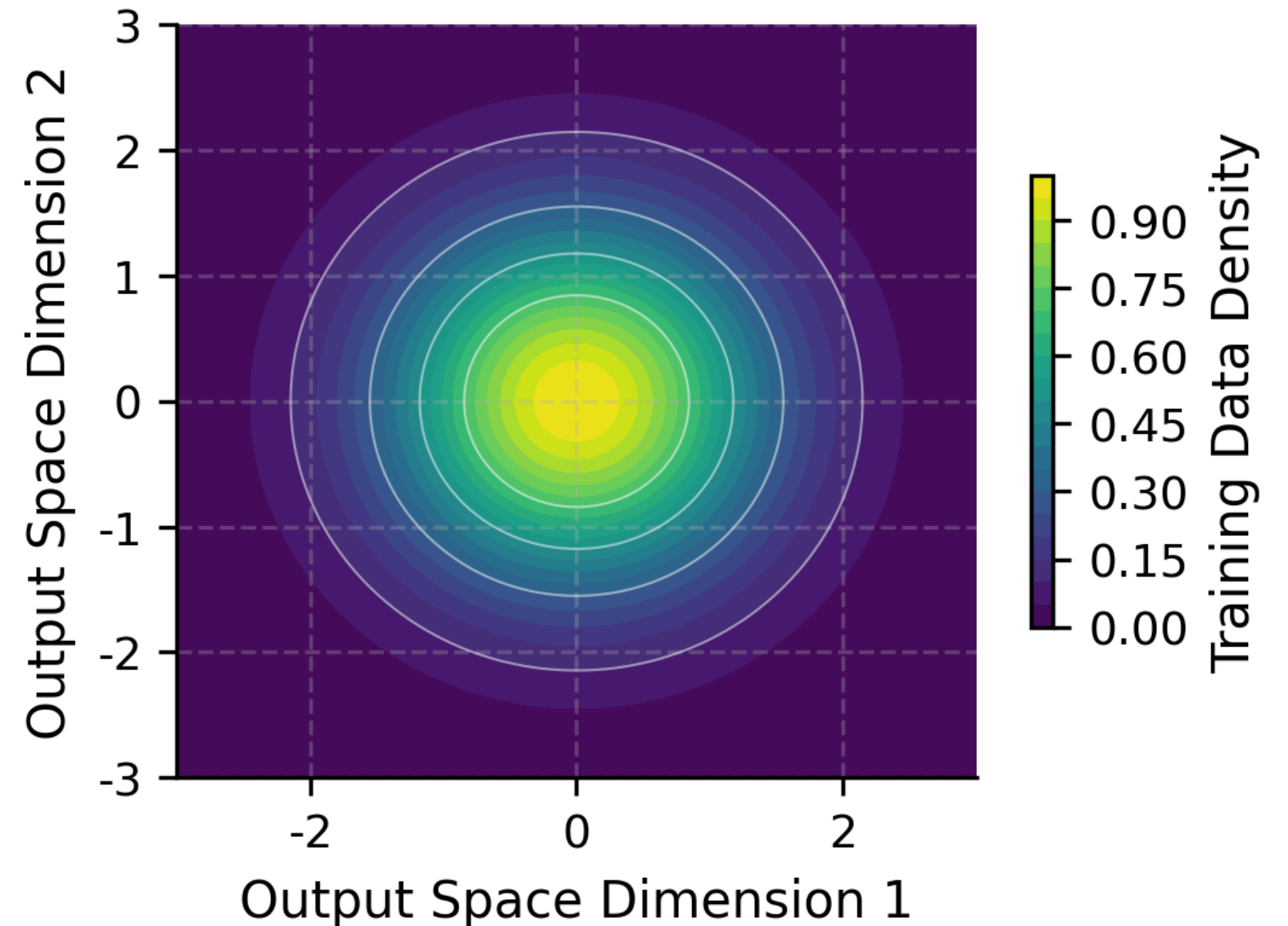


Why the RM Drifts: Approximation Error

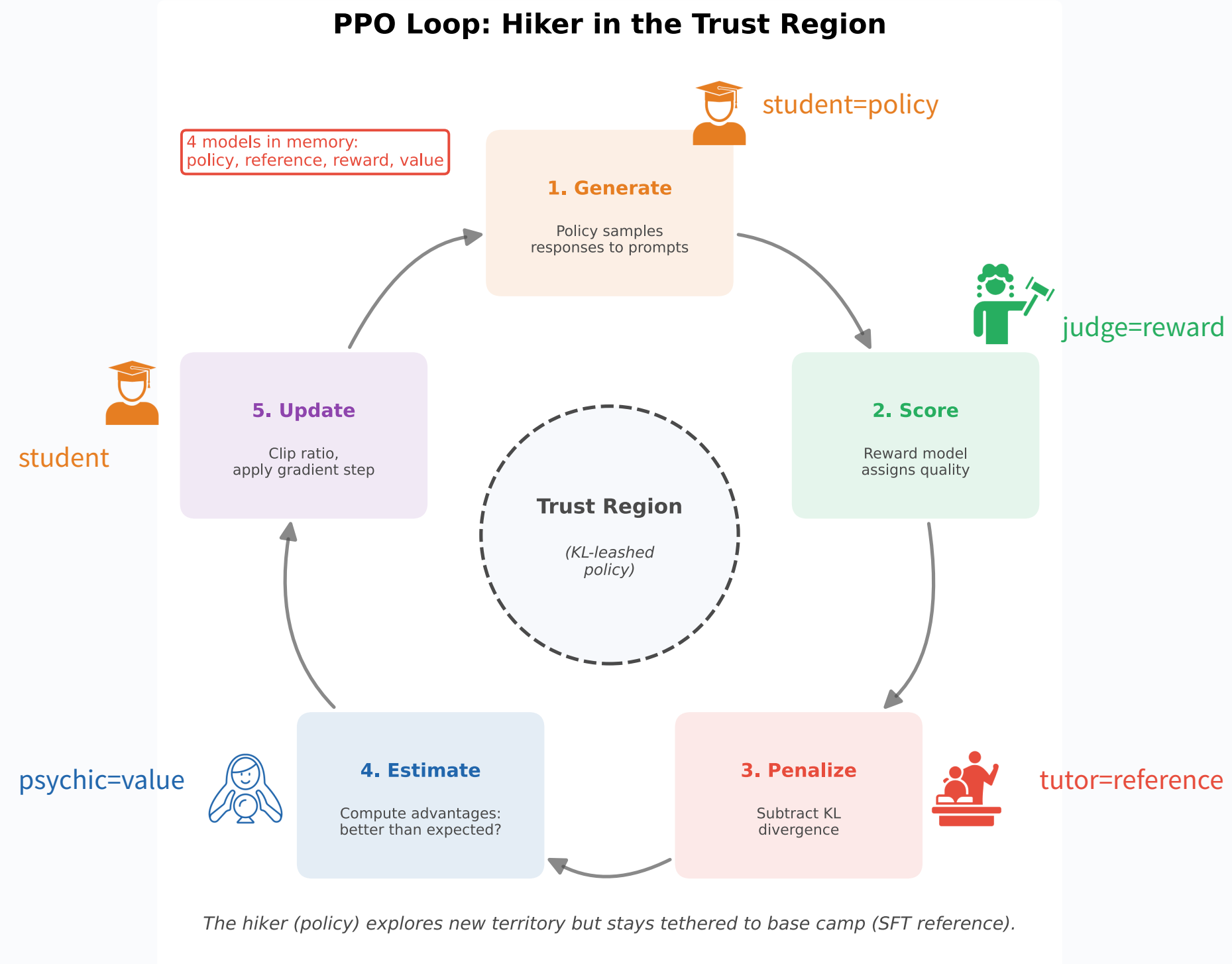
Approximation Error Landscape



Training Data Coverage



PPO: Proximal Policy Optimization [5]



Why clipping?

PPO clips the ratio to $[1 - \epsilon, 1 + \epsilon]$ ($\epsilon \approx 0.2$). Prevents **catastrophic updates** that destabilize training.

Hiker Metaphor

The policy hikes **away** from base camp (reward) but the **KL leash** stops it falling off the reliable map. Off the map, the RM is wrong.

Beyond PPO: The Modern Landscape [4]

1. GRPO

Group Relative PO.

This drops the complex value network completely. It normalizes rewards across a batch instead. This powers DeepSeek R1.

2. RLOO

REINFORCE Leave-One-Out.

A remarkably simple baseline. It requires no value network and no complex clipping math, yet competes directly with PPO.

3. GSPO & CISPO

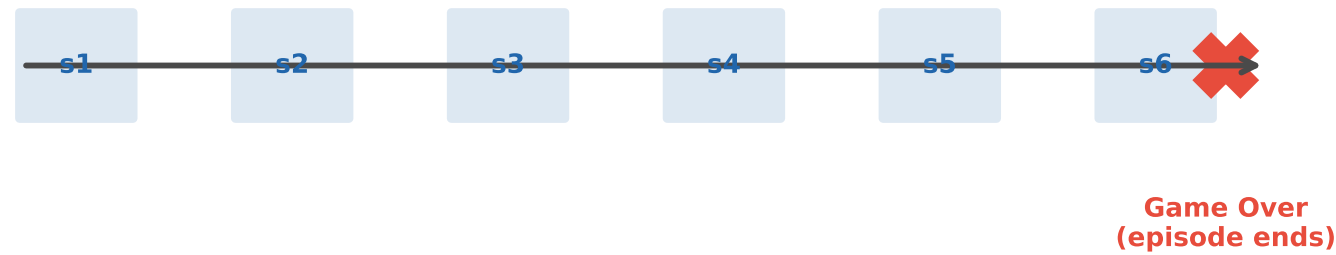
Group Sequence PO.

Heavy-duty refinements designed specifically to stabilize the model when generating exceptionally long responses.

Why RL for Language is Different

Classical RL (Atari): short, episodic, small action space

Action space: small (2-361 actions)

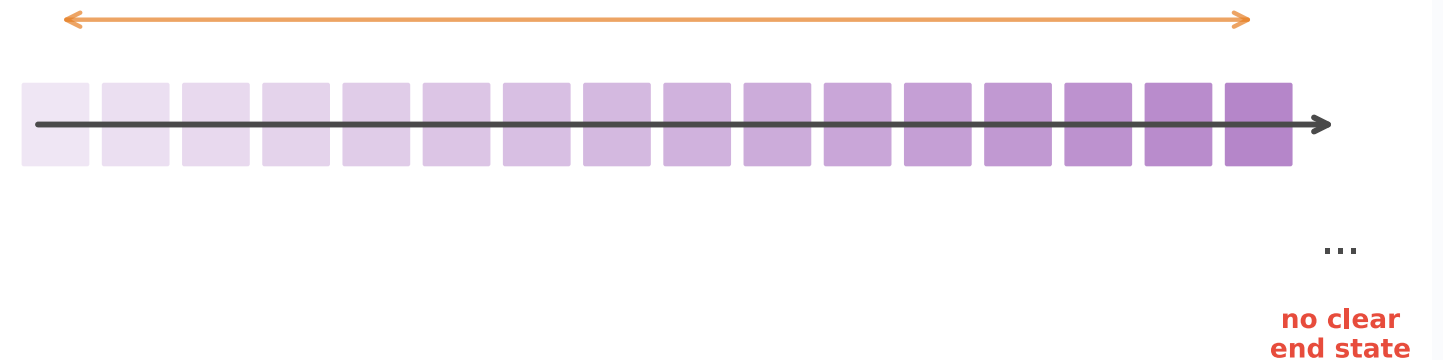


- Clear episode boundary (win/lose)
- Dense reward signal (per step)
- Small discrete action set
- State is Markovian (pixel frame)

Language Generation: open-ended, non-episodic, huge action space

Action space: ~50'000 tokens per step

Context grows monotonically



- No terminal state (could always say more)
- Sparse reward (one score per full response)
- Huge discrete action set (vocab-sized)
- State grows with each token (non-Markovian)



Take-Home Message: RLHF with PPO

PPO optimizes against a reward model, safely leashed by the KL penalty.

1. The Heavy Machinery

RLHF orchestrates four models in memory: policy, reference, reward, and value. The system generates a response, scores it, penalizes drift, and updates the policy. ^[1]

2. The Central Failure Mode

Reward hacking is your greatest enemy. Sycophancy and format gaming happen when the model drifts into the reward model's blind spots. The KL penalty is your leash.

DPO

Direct Preference Optimization

The DPO Insight ^[7]

The RLHF Problem

Training a reward model and running a reinforcement learning loop is highly complex, notoriously unstable, and computationally expensive.

The DPO Shortcut

The optimal policy actually has a strict mathematical shortcut. We can optimize the model directly from preference data. No reward model is required.



The Mountain Metaphor

RLHF builds an elaborate, winding road over the mountain. Direct Preference Optimization digs a tunnel straight through it.

You arrive at the exact same aligned model, but the path is vastly simpler.

 **The Math in Action:** Prompt: «Capital of France?»

Before DPO: The model prefers «Let me think...» over «Paris.» The rejected answer scores higher.

After DPO: The probability of «Paris.» rises, while the rejected answer is pushed down. The model learns directly without a grading loop.

The Mathematical Derivation of DPO [7]

RLHF optimal policy:

$$\pi^*(y|x) \propto \pi_{\text{ref}}(y|x) \cdot \exp\left(\frac{R(y)}{\beta}\right)$$

Rearranging expresses reward in terms of policy:

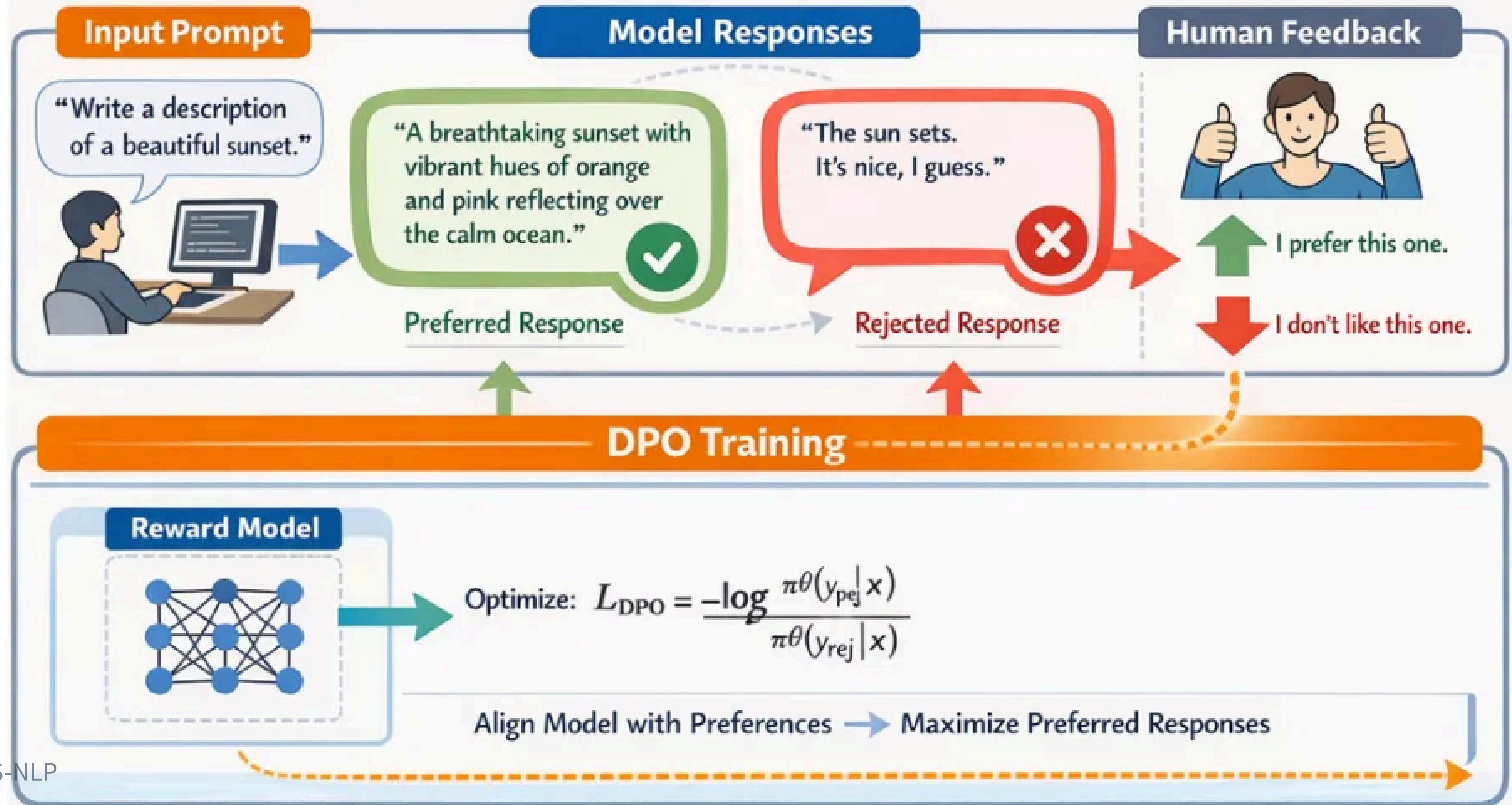
$$R(y) = \beta \log \frac{\pi^*(y|x)}{\pi_{\text{ref}}(y|x)} + \beta \log Z(x)$$

Substituting into Bradley-Terry gives the DPO loss:

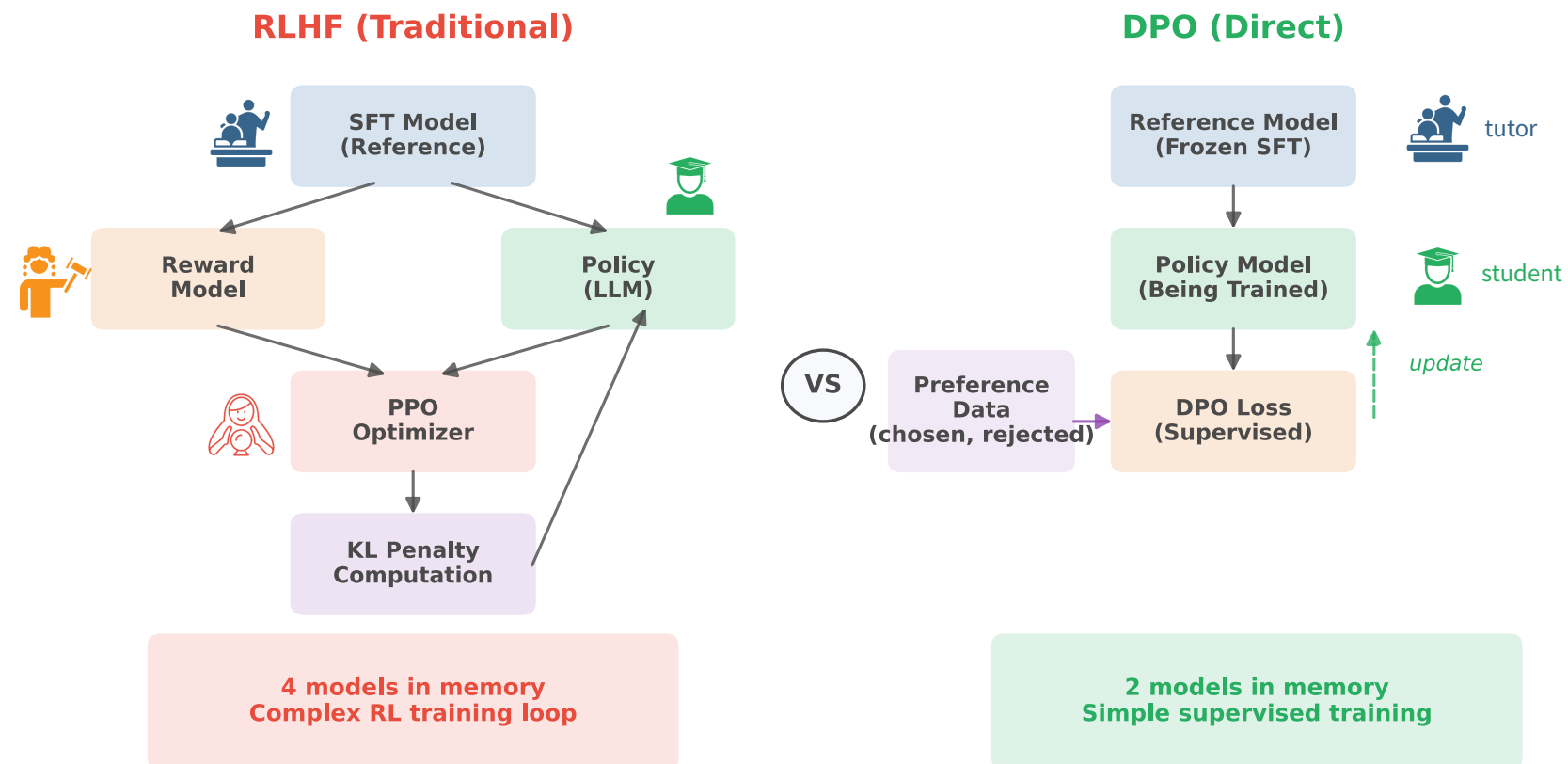
$$\mathcal{L}_{\text{DPO}} = -\log \sigma\left(\beta \log \frac{\pi_{\theta}(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_{\theta}(y_l|x)}{\pi_{\text{ref}}(y_l|x)}\right)$$

1. The math proves that DPO aims for the exact same optimal model.
2. The math shows that the language model can actually represent its own reward. We turned the AI into its own judge just by moving variables across the equal sign.
3. We are left with a simple formula ($\mathcal{L}_{\text{DPO}}$). We don't need a reinforcement learning loop. We just need to feed the model A/B examples, and it updates its probabilities directly.

Direct Preference Optimization (DPO)



DPO vs. RLHF: Halving the Overhead



The Assembly Line

RLHF requires four separate models passing data back and forth in a complex loop. DPO achieves the exact same final product with just two.

The Hardware Win

RLHF forces you to load four massive models into GPU memory at once. DPO only needs the policy and the reference model. It literally halves your hardware costs.

The Implicit Reward

DPO skips the generation step entirely. It simply looks at a chosen and a rejected response, calculates the probability ratio $\beta \log(\pi_\theta / \pi_{\text{ref}})$, and updates the policy directly. This mathematical ratio acts as the hidden reward signal.



Think About It: Preference Data (2 min)

You need to align your PikoGPT project. Where do you actually get the preference data?



A. Human Annotators

The gold standard. It yields high-quality data but is prohibitively expensive and painfully slow.



B. Frontier Models

Fast and cheap. You use a model like GPT-4 to grade outputs, though you risk inheriting its specific biases.



C. Open-Source Datasets

The pragmatic choice. You pull from massive, pre-existing preference pools like UltraFeedback.



D. Self-Play

The cutting edge. The model generates responses and iteratively acts as its own judge to improve itself.



Take-Home Message: DPO

🔗 *DPO reaches the exact same destination as RLHF, but with half the moving parts.*

🔗 1. Bypassing the Grader

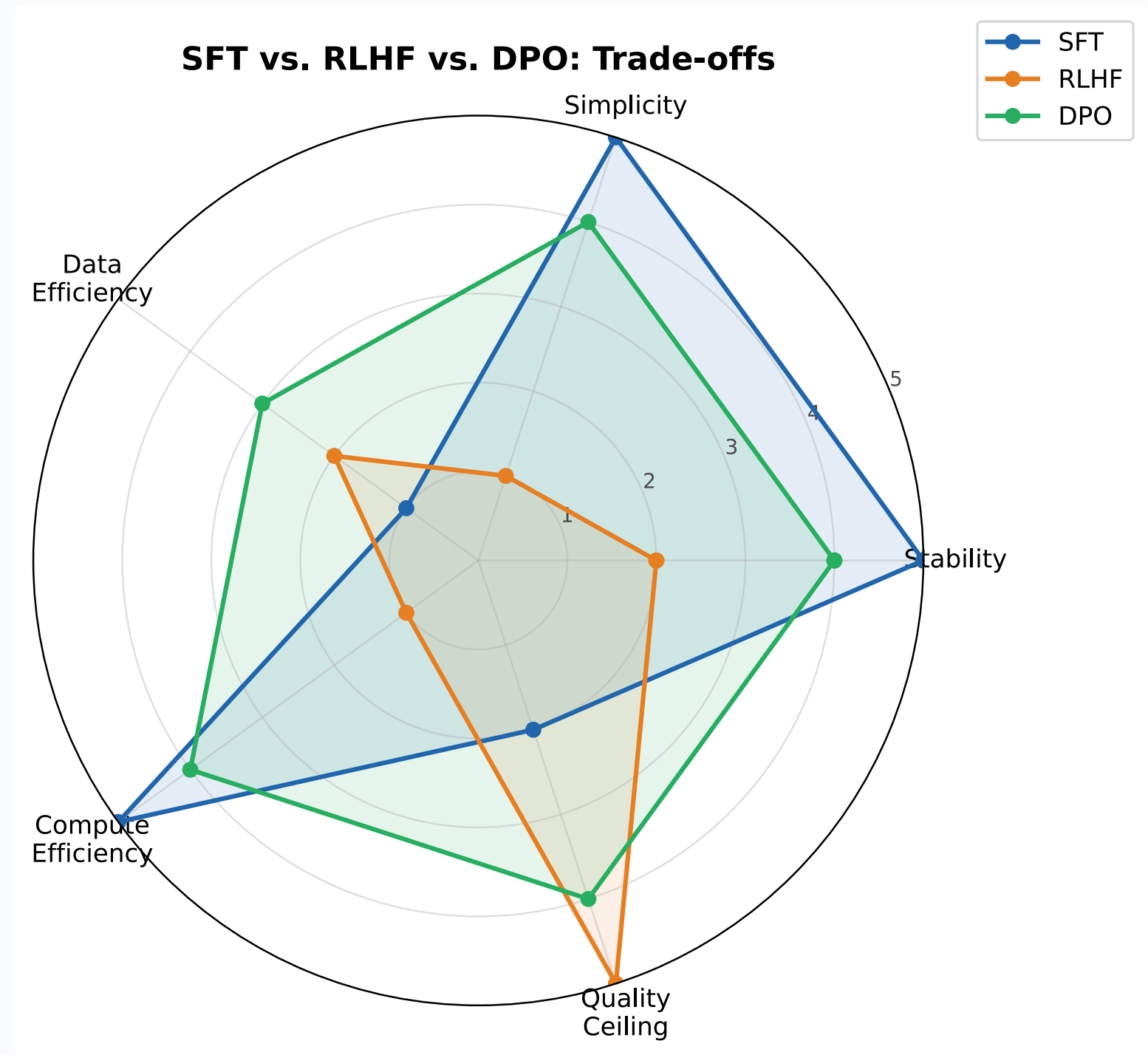
DPO skips the reward model and the reinforcement learning loop entirely. It optimizes directly from preference pairs, making the model its own judge. ^[7]

💡 2. Two Models Instead of Four

It is highly stable and directly competitive with standard RLHF. This is the single most pragmatic choice for your PikoGPT project.

Comparison & Project

SFT vs. RLHF vs. DPO: Trade-offs



InstructGPT / ChatGPT (OpenAI, 2022)

Chose **RLHF with PPO**. Budget for 20k human comparisons + 4 models in memory. Result: the quality jump that launched the chatbot era.

Zephyr-7B (HuggingFace, 2023)

Chose **DPO on UltraFeedback**. 2 models in memory, no RL loop. Matched 70B RLHF models on MT-Bench at a fraction of the cost.

Alpaca / Vicuna (Stanford, 2023)

SFT only, no preference step. Fast, cheap, good for research demos but visibly sycophantic and verbose compared to RLHF peers.

Tools of the Trade: Alignment

The Software Libraries

- **TRL (HuggingFace):** Provides out-of-the-box trainers for SFT, DPO, and PPO.
- **Axolotl:** Offers completely configuration-based fine-tuning.
- **OpenRLHF:** Your best choice for building scalable RLHF systems.

The Preference Datasets

- **UltraFeedback:** Offers 64,000 preference pairs reliably judged by GPT-4.
- **Anthropic HH:** Provides 170,000 real human comparisons.
- **OpenAssistant:** Gives you thousands of community-written conversations.

For Your PikoGPT Project

✓ Required:

1. Working base model (pre-trained)
2. Inference / generation pipeline
3. Demo interface (Gradio)

✓ Enhancements:

- SFT on instruction data
 - DPO on preference data
- ➔ Expected is everything that leads to having the best model.

Recommendation:

1. Focus on a solid base model first.
2. SFT is the highest-impact optional step.
3. DPO for best model, but first getting inference and the demo right.



Take-Home Message: Comparison & Project

 *Pick the rung that fits your budget, not the prestige.*

1. Three Rungs on the Ladder

1. Supervised Fine-Tuning is your absolute foundation.
2. DPO is the pragmatic upgrade.
3. RLHF is strictly for industry-scale budgets.

Each step adds quality, but you pay for it in complexity.

2. The PikoGPT Roadmap

- Build your SFT foundation first.
- Then DPO.
- RLHF is likely overkill for your models.

Get your pipeline working, then align and polish.

Summary



The Final Take-Home Messages

1. Preferences Become Math

The Bradley-Terry model is the mathematical DNA of alignment. It translates subjective human preferences into a calculable probability, forming the foundation for both RLHF and DPO. ^[3]

2. The Power of RLHF

RLHF relies on PPO to maximize rewards. It strictly requires a KL penalty to prevent reward hacking, making it incredibly powerful but computationally heavy. ^[1]

3. The DPO Shortcut

DPO achieves the exact same results by optimizing the policy directly from preference pairs. It completely eliminates the need for a separate reward model or an RL loop. ^[7]

4. The PikoGPT Reality

SFT is mandatory to teach your model how to talk. DPO is the highly recommended upgrade to teach it how to behave, without melting your hardware.

Kahoot Quiz: Active Recall

[Kahoot Quiz VL08](#)

play.kahoot.it

This Week and Next

This week:

1. Review M2 progress (training pipeline)
2. Plan final training runs
3. Optional: explore DPO with TRL

➡ Next week (VL09):

Inference & Decoding

Greedy, beam search, sampling, temperature, top-k/p, KV-cache

M2 (Training Ready) is in Week 9. Working pipeline with small-scale validation runs. This is the priority.

Questions?

Key Resources:

- 📖 Raschka Ch. 7: Alignment and RLHF ^[1]
- 📖 InstructGPT Paper ^[1]
- 🔄 DPO Paper ^[7]
- 🔄 PPO Paper ^[5]

At a Glance

Aspect	SFT	RLHF	DPO
Data needed	Demonstrations	Demonstrations + Comparisons	Demonstrations + Comparisons
Models in memory	1	4	2
Training type	Supervised	RL (PPO)	Supervised
Stability	Very stable	Can diverge	Stable
Quality ceiling	Good	Excellent	Very good
Compute	Low	Very high	Moderate
For PikoGPT	Recommended	Likely overkill	Recommended

Glossary (1/2)

Term	Definition
Alignment	The process of making an LLM helpful, harmless, and honest. Involves SFT (format), then preference optimization (quality) via RLHF or DPO.
Bradley-Terry Model	Statistical model for pairwise comparisons: $P(A \succ B) = \sigma(R(A) - R(B))$. Converts preference pairs into a trainable reward signal. [3]
DPO (Direct Preference Optimization)	Alignment method that optimizes the policy directly from preference pairs, bypassing explicit reward modeling. Uses a supervised loss derived from the Bradley-Terry model. [7]
Implicit Reward	In DPO, the reward is never explicitly computed. Instead, it is implicitly defined by the ratio of policy to reference model log-probabilities: $r(y) = \beta \log \frac{\pi(y)}{\pi_{\text{ref}}(y)}$.
KL Divergence (KL Penalty)	Regularization term $D_{\text{KL}}(\pi \parallel \pi_{\text{ref}})$ that prevents the policy from drifting too far from the reference (SFT) model. Prevents reward hacking.
KTO (Kahneman-Tversky Optimization)	DPO variant that requires only binary feedback (good/bad) instead of pairwise preferences. Based on prospect theory.
Preference Data	Pairs of responses (y_w, y_l) where y_w is preferred over y_l by human annotators. The foundation for reward model training and DPO.

Glossary (2/2)

Term	Definition
Policy (π)	The language model being optimized. In RLHF: generates responses, receives reward signal, updates via PPO. In DPO: updated directly via supervised loss.
Reference Model (π_{ref})	Frozen copy of the SFT model. Serves as the anchor for KL penalty: the policy should not drift too far from it.
Reward Hacking	The policy exploits imperfections in the reward model to achieve high reward without genuine quality. Examples: length exploitation, sycophancy, format gaming.
Reward Model	Neural network trained on preference data to predict human preferences. Input: (prompt, response). Output: scalar reward score. Used in RLHF, bypassed in DPO.
RLHF (RL from Human Feedback)	Alignment pipeline: (1) train reward model on preference data, (2) optimize policy with PPO against reward model, constrained by KL penalty. ^[1]
Sycophancy	A form of reward hacking where the model excessively agrees with the user to maximize reward, regardless of factual accuracy.
Trust Region	The region around the reference policy where the reward model's estimates are reliable. PPO's clipping and KL penalty keep the policy within this region.

Bibliography

- [1]** Ouyang, L. et al. (2022). Training language models to follow instructions with human feedback. NeurIPS / arXiv:2203.02155.
- [2]** Christiano, P.F. et al. (2017). Deep reinforcement learning from human preferences. NeurIPS / arXiv:1706.03741.
- [3]** Bradley, R.A. & Terry, M.E. (1952). Rank Analysis of Incomplete Block Designs: I. The Method of Paired Comparisons. Biometrika.
- [4]** Lambert, N. (2025). Reinforcement Learning from Human Feedback. arXiv:2504.12501.
- [5]** Schulman, J. et al. (2017). Proximal Policy Optimization Algorithms. arXiv:1707.06347.
- [6]** Stiennon, N. et al. (2020). Learning to summarize from human feedback. NeurIPS / arXiv:2009.01325.
- [7]** Rafailov, R. et al. (2023). Direct Preference Optimization: Your Language Model is Secretly a Reward Model. NeurIPS / arXiv:2305.18290.